

Dictionnaire complet des API EBAC

Version : API v1

Base URL : <https://api-ebac.severinzran.ci/api/v1>

Format : JSON UTF-8, sauf téléversement de photo (multipart/form-data).

1. Conventions générales

Routes protégées :

Accept: application/json

Content-Type: application/json

Authorization: Bearer {token}

Codes HTTP : 200 succès, 201 création, 202 OTP envoyé, 401 non authentifié, 403 accès refusé, 404 ressource absente, 422 validation, 429 limite atteinte, 500 erreur serveur.

Erreur de validation standard :

```
{
  "message": "Les données fournies sont invalides.",
  "errors": {
    "email": ["Le champ email est obligatoire."]
  }
}
```

Les codes USR-, PER- et MEN- sont générés par l'API. Le code du rôle est obligatoirement saisi par le frontend.

2. Dictionnaire des objets

2.1 Utilisateur

Champ	Type	Null	Source	Description
---	---	---	---	---
`id`	entier	non	API	Identifiant du compte
`civillite_id`	entier	oui	Front	Référence à la civilité
`matricule`	chaîne	oui	API	Matricule automatique au format `EBAC-0000-ANNEE`, par exemple `EBAC-0001-2026`
`code`	chaîne	non	API	Code automatique du nouveau compte
`user_code`	chaîne	oui	API	Code de l'utilisateur créateur
`user_id`	entier	oui	API	ID de l'utilisateur créateur
`nom`	chaîne(150)	non	Front	Nom
`prenoms`	chaîne(150)	non	Front	Prénoms
`email`	email(150)	non	Front	Email unique
`photo`	chaîne	oui	API	Chemin de la photo
`photo_url`	URL	oui	API	URL publique de la photo
`id_role`	entier	non	Front	Rôle attribué
`is_active`	booléen	non	Front	Compte activé
`statut`	enum	non	Front	`Actif`, `Suspendu`, `Bloqué`, `Désactivé`
`deux_fa_active`	booléen	non	Front	Double authentification active
`tentatives_echouees`	entier	non	API	Échecs de connexion
`cree_le`	ISO-8601	oui	API	Date de création
`derniere_connexion`	ISO-8601	oui	API	Dernière connexion
`role`	objet	oui	API	`{id, code, libelle}`
`civillite`	objet	oui	API	`{id, code, name, abreviation}`

2.2 Rôle

Champ	Type	Source	Description
---	---	---	---
`id`	entier	API	Identifiant
`code`	chaîne(30)	Front	Obligatoire, unique, normalisé en majuscules
`libelle`	chaîne(80)	Front	Nom du rôle
`description`	chaîne(255)	Front	Description facultative
`actif`	booléen	Front	Active ou désactive le rôle
`permissions`	tableau	Front/API	Permissions associées
`autorisations`	tableau	Front/API	Couples `menu_id` et `action_ids` cochés
`permissions_count`	entier	API	Nombre de permissions

2.3 Permission

Champ	Type	Source	Description
---	---	---	---
`id`	entier	API	Identifiant
`code`	chaîne	API	Automatique : `PER-000001`
`libelle`	chaîne(120)	Front	Nom de la permission
`description`	chaîne(255)	Front	Description facultative
`actions`	tableau	Front/API	Actions autorisées
`roles`	tableau	API	Rôles utilisant la permission

2.4 Action

Champ	Type	Source	Description
---	---	---	---
`id`	entier	API	Identifiant
`code`	chaîne(30)	API	Généré depuis le libellé
`libelle`	chaîne(120)	Front	Nom de l'action
`description`	chaîne(255)	Front	Description facultative
`actif`	booléen	Front	État de l'action
`permissions_count`	entier	API	Nombre de permissions associées

Actions initiales : AJOUTER, SUPPRIMER, MODIFIER, VOIR, IMPRIMER, TELECHARGER.

2.5 Menu

Champ	Type	Source	Description
---	---	---	---
`id`	entier	API	Identifiant
`id_parent`	entier/null	Front	Menu parent
`code`	chaîne	API	Automatique : `MEN-000001`
`libelle`	chaîne(150)	Front	Texte du sidebar
`description`	chaîne(255)	Front	Description
`route`	chaîne(180)	Front	Route frontend
`route_active`	chaîne(180)	Front	Motif de route active
`icone`	chaîne(100)	Front	Nom de l'icône
`groupe`	chaîne(100)	Front	Groupe d'affichage
`ordre`	entier 0..65535	Front	Ordre d'affichage
`visible`	booléen	Front	Visible dans le sidebar
`actif`	booléen	Front	Menu actif
`permissions`	tableau	Front/API	Permissions nécessaires
`actions`	tableau	Front/API	Actions proposées pour ce module
`enfants`	tableau	API	Sous-menus

2.6 Tables de liaison des autorisations

Table	Clé	Description
---	---	---
`role_permissions`	`id_role + id_permission`	Permissions métier attribuées au rôle
`permission_actions`	`id_permission + id_action`	Actions rattachées à une permission
`menu_permissions`	`id_menu + id_permission`	Permissions donnant accès au menu
`menu_actions`	`id_menu + id_action`	Actions disponibles dans la ligne du module
`role_menu_actions`	`id_role + id_menu + id_action`	Cases cochées de la matrice du rôle

2.7 Pagination

```
{
  "current_page": 1,
  "data": [],
  "first_page_url": "https://api-ebac.severinzran.ci/api/v1/administration/roles?page=1",
  "from": 1,
  "last_page": 1,
  "last_page_url": "...page=1",
  "links": [],
  "next_page_url": null,
  "path": ".../administration/roles",
  "per_page": 15,
  "prev_page_url": null,
  "to": 1,
  "total": 1
}
```

3. Authentication

POST `/auth/connexion`

Public. Limite : 5 requêtes/minute.

```
{
  "email": "admin@ebac.ci",
  "password": "password",
  "nom_appareil": "Frontend EBAC"
}
```

Réponse 202 :

```
{
  "message": "Un code OTP a été envoyé à votre adresse e-mail.",
}
```

```
"otp_requis": true,
"id_tentative": 25,
"expire_dans": 600
}
```

POST `/auth/confirmer-otp`

```
{"id_tentative": 25, "code_otp": "123456"}
```

Réponse 200 :

```
{
  "message": "Connexion réussie.",
  "token": "1|jeton-sanctum",
  "token_type": "Bearer",
  "redirect_to": "/dashboard/index",
  "utilisateur": {
    "id": 1,
    "code": "USR-000001",
    "nom": "Zran",
    "prenoms": "Severin",
    "email": "admin@ebac.ci",
    "id_role": 1,
    "is_active": true,
    "statut": "Actif",
    "role": {"id": 1, "code": "ADMIN", "libelle": "Administrateur"}
  }
}
```

GET `/auth/profil`

Protégé. Réponse 200 : {"utilisateur": {...objet Utilisateur...}}.

POST `/auth/modifier-mot-de-passe`

Protégé. Appelé volontairement par un utilisateur connecté.

```
{"mot_de_passe_actuel":"PasswordActuel123","password":"NouveauPassword456","password_confirmation":"NouveauPassword456"}
```

Réponse 200 : {"message":"Votre mot de passe a été modifié avec succès. Votre session reste active.,"deconnexionrequis":false}.

Le nouveau mot de passe doit différer du mot de passe actuel. La session reste active et l'utilisateur choisit lui-même quand se déconnecter. Sa connexion suivante exige l'OTP.

Après une réinitialisation via « mot de passe oublié », et uniquement dans ce cas, la première connexion est autorisée sans OTP une seule fois.

POST `/auth/deconnexion`

Protégé. Réponse 200 :

```
{"message": "Déconnexion réussie."}
```

POST `/auth/deconnexion-globale`

Protégé. Supprime tous les jetons. Réponse 200 :

```
{"message": "Déconnexion de tous les appareils réussie."}
```

4. Mot de passe oublié

POST `/auth/mot-de-passe-oublie`

```
{"email": "utilisateur@ebac.ci"}
```

Réponse 200 identique que l'email existe ou non :

```
{"message": "Si cette adresse existe, un code de réinitialisation a été envoyé."}
```

POST `/auth/verifier-code-reinitialisation`

```
{"email": "utilisateur@ebac.ci", "code": "915370"}
```

Réponse 200 :

```
{  
  "message": "Code vérifié avec succès.",  
  "reset_token": "jeton-temporaire",  
  "expire_dans": 600  
}
```

POST `/auth/reinitialiser-mot-de-passe`

```
{  
  "email": "utilisateur@ebac.ci",  
  "reset_token": "jeton-temporaire",  
  "password": "NouveauMotDePasse123!",  
  "password_confirmation": "NouveauMotDePasse123!"  
}
```

Réponse 200 :

```
{"message": "Votre mot de passe a été réinitialisé avec succès. Vous pouvez maintenant vous connecter."}
```

5. Comptes utilisateurs — permission `COMPTE\_GERER`

GET `/administration/comptes`

Query : recherche, statut, idrole, isactive, parpage, page.

Réponse 200 : pagination dont data contient des objets Utilisateur.

GET `/administration/comptes/create`

Réponse 200 :

```
{
  "roles": [{ "id": 1, "code": "ADMIN", "libelle": "Administrateur" }],
  "civilites": [{ "id": 1, "code": "M", "name": "Monsieur", "abreviation": "M." }],
  "statuts": ["Actif", "Suspendu", "Bloqué", "Désactivé"]
}
```

POST `/administration/comptes`

Utiliser multipart/form-data si photo est présente.

Champ	Règle

```
`civilite_id` | obligatoire, entier, existe dans `civilite.id`  
`nom`, `prenoms` | obligatoires, chaîne, max 150  
`email` | obligatoire, email unique, max 150  
`id_role` | obligatoire, rôle existant  
`photo` | jpg/jpeg/png/webp, max 2 Mo  
`is_active`, `deux_fa_active` | booléens facultatifs  
`statut` | enum facultatif  
`code`, `user_code`, `user_id` | interdits ; calculés par l'API
```

Réponse 201 :

```
{
  "message": "Compte créé avec succès. Les identifiants temporaires ont été envoyés par e-mail.",
  "compte": { "id": 12, "matricule": "EBAC-0001-2026", "code": "USR-000012", "user_id": 1,
    "user_code": "USR-ADMIN", "nom": "Zran", "prenoms": "Severin", "email": "severin@ebac.ci",
    "photo_url": "https://api-ebac.severinzran.ci/api/v1/utilisateurs/12/photo" }
}
```


GET `/administration/comptes/{id}`

Réponse 200 : {"utilisateur": {...objet Utilisateur avec rôle et civilité...}}.

GET `/administration/comptes/{id}/edit`

Réponse 200 : utilisateur, rôles, civilités et statuts disponibles.

PUT/PATCH/POST `/administration/comptes/{id}`

Champs de création facultatifs, plus password (min 8) et passwordconfirmation.

Pour remplacer une photo, utiliser obligatoirement POST multipart/form-data avec le champ fichier photo. Cette méthode évite la limitation PHP sur les fichiers envoyés directement en PUT/PATCH.

Réponse 200 : {"message": "Compte modifié avec succès.", "compte": {...}}.

GET `/utilisateurs/{id}/photo`

URL publique retournant directement le fichier image. Cette URL est fournie dans photourl et ne dépend pas d'un lien symbolique public/storage sur cPanel. Réponse 404 si aucune photo n'existe.

DELETE `/administration/comptes/{id}`

Réponse 200 : {"message": "Compte supprimé avec succès."}.

Réponse 422 si l'utilisateur tente de supprimer son propre compte.

6. Profil utilisateur

GET `/administration/profil`

Réponse 200 : {"utilisateur": {...}}.

GET `/administration/profil/edit`

Réponse 200 : utilisateur et civilités disponibles.

PUT/PATCH/POST `/administration/profil`

Champs : civiliteid, nom, prenom, email, photo, motdepasseactuel, password, passwordconfirmation. Le mot de passe actuel est obligatoire si un nouveau mot de passe est envoyé.

Pour modifier la photo, utiliser POST multipart/form-data.

Réponse 200 : {"message":"Profil modifié avec succès.","utilisateur":{...}}.

7. Rôles — permission `ROLE\_GERER`

GET `/administration/roles`

Query : recherche, parpage, page. Réponse 200 : pagination avec rôles, permissions et permissionscount.

POST `/administration/roles`

```
{"code":"GESTIONNAIRE","libelle":"Gestionnaire","description":"Gestion des étudiants","actif":true,"permission_ids":[1,2],"autorisations":[{"menu_id":1,"action_ids":[1,3,4]}]}
```

Réponse 201 :

```
{"message":"Rôle créé avec succès.","role":{"id":2,"code":"GESTIONNAIRE","libelle":"Gestionnaire","description":"Gestion des étudiants","permissions":[{"id":1,"code":"PER-000001","libelle":"Étudiants"}]}}
```

GET `/administration/roles/{id}`

Réponse 200 : {"role":{...rôle avec permissions...}}.

PUT/PATCH `/administration/roles/{id}`

Champs : code, libelle, description, permissionids. À la création, code est obligatoire, unique, limité à 30 caractères et converti en majuscules.

Réponse 200 : {"message":"Rôle modifié avec succès","role":{"...}}.

PUT `/administration/roles/{id}/permissions`

```
{"permission_ids":[1,3,5]}
```

Réponse 200 : {"message":"Permissions du rôle mises à jour","role":{"...rôle avec permissions...}}.

GET `/administration/roles/matrice-autorisations`

Réponse 200 : {"modules":[{"menuid":1,"code":"MEN-000001","libelle":"Étudiants","actions":[{"id":1,"code":"VOIR","libelle":"Voir","selectionnee":false}]}]}.

GET `/administration/roles/{id}/matrice-autorisations`

Même matrice avec selectionnee: true pour les cases déjà attribuées au rôle.

PUT `/administration/roles/{id}/autorisations`

```
{"autorisations":[{"menu_id":1,"action_ids":[1,3,4]},{"menu_id":2,"action_ids":[4]}]}
```

Cette requête remplace toute la matrice du rôle.

DELETE `/administration/roles/{id}`

Réponse 200 : {"message":"Rôle supprimé avec succès."}. ADMIN et un rôle attribué à des comptes ne peuvent pas être supprimés (422).

8. Permissions — permission `PERMISSION\_GERER`

GET `/administration/permissions`

Query : recherche, parpage, page. Réponse 200 : pagination de permissions.

POST `/administration/permissions`

```
{"libelle":"Gestion des étudiants","description":"Accès au module étudiants"}
```

Réponse 201 :

```
{"message":"Permission créée avec succès.","permission":{"id":1,"code":"PER-000001","libelle":"Gestion des étudiants","description":"Accès au module étudiants"}}
```

GET `/administration/permissions/{id}`

Réponse 200 :

```
{"permission":{"id":1,"code":"PER-000001","libelle":"Gestion des étudiants","roles":[],"actions":[{"id":1,"code":"AJOUTER","libelle":"Ajouter","actif":true}]}}
```

PUT/PATCH `/administration/permissions/{id}`

Champs : libelle, description. code interdit.

Réponse 200 : {"message":"Permission modifiée avec succès.","permission":{...}}.

PUT `/administration/permissions/{id}/actions`

```
{"action_ids":[1,3,4,5]}
```

Réponse 200 : {"message":"Actions de la permission mises à jour.","permission":{...permission avec actions...}}.

DELETE `/administration/permissions/{id}`

Réponse 200 : {"message":"Permission supprimée avec succès."}. Réponse 422 si elle est encore attribuée à un rôle.

9. Actions — permission `ACTION\_GERER`

GET `/administration/actions`

Réponse 200 :

```
{"actions":[{"id":1,"code":"AJOUTER","libelle":"Ajouter","description":"Créer un nouvel  
enregistrement.", "actif":true, "permissions_count":0}]}
```

POST `/administration/actions`

```
{"libelle":"Valider","description":"Valider un dossier","actif":true}
```

Réponse 201 : {"message":"Action créée avec succès","action":{"id":7,"code":"VALIDER","libelle":"Valider","description":"Valider un dossier","actif":true}}.

GET `/administration/actions/{id}`

Réponse 200 : {"action":{...action avec permissions...}}.

PUT/PATCH `/administration/actions/{id}`

Champs : libelle, description, actif. code interdit et reste immuable.

Réponse 200 : {"message":"Action modifiée avec succès","action":{...}}.

DELETE `/administration/actions/{id}`

Réponse 200 : {"message":"Action supprimée avec succès."}. Réponse 422 si elle est attribuée à une permission.

10. Menus — permission `MENU\_GERER`

GET `/administration/menus`

Réponse 200 : {"menus":[{"...menu avec permissions...}]}.

POST `/administration/menus`

```
{
  "id_parent": null,
  "libelle": "Étudiants",
  "description": "Gestion des étudiants",
  "route": "/etudiants",
  "route_active": "/etudiants*",
  "icone": "users",
  "groupe": "Scolarité",
  "ordre": 10,
  "visible": true,
  "actif": true,
  "permission_ids": [1],
  "action_ids": [1, 2, 3, 4]
}
```

Réponse 201 : {"message":"Menu créé avec succès.","menu":{"id":1,"code":"MEN-000001","libelle":"Étudiants","route":"/etudiants","permissions":[...]}}.

GET `/administration/menus/{id}`

Réponse 200 : {"menu":{"...menu avec permissions et enfants...}}.

PUT/PATCH `/administration/menus/{id}`

Tous les champs de création sont facultatifs. code interdit. idparent ne peut pas désigner le menu lui-même.

Réponse 200 : {"message":"Menu modifié avec succès.,"menu":{...}}.

DELETE `/administration/menus/{id}`

Réponse 200 : {"message":"Menu supprimé avec succès."}.

11. Navigation

GET `/navigation/sidebar`

Protégé. Retourne uniquement les menus actifs et visibles autorisés par le rôle de l'utilisateur.

```
{
  "menus": [
    {
      "id": 1,
      "code": "MEN-000001",
      "libelle": "Étudiants",
      "route": "/etudiants",
      "route_active": "/etudiants*",
      "icone": "users",
      "groupe": "Scolarité",
      "ordre": 10,
      "enfants": []
    }
  ]
}
```

12. Catalogue exhaustif des URL

Méthode	URL
POST	`/auth/connexion`
POST	`/auth/confirmer-otp`
POST	`/auth/modifier-mot-de-passe`
POST	`/auth/mot-de-passe-oublie`
POST	`/auth/verifier-code-reinitialisation`
POST	`/auth/reinitialiser-mot-de-passe`
GET	`/auth/profil`
POST	`/auth/deconnexion`
POST	`/auth/deconnexion-globale`
GET	`/navigation/sidebar`
GET, POST	`/administration/comptes`
GET	`/administration/comptes/create`
GET, PUT, PATCH, POST, DELETE	`/administration/comptes/{id}`
GET	`/administration/comptes/{id}/edit`
GET	`/utilisateurs/{id}/photo`
GET, PUT, PATCH, POST	`/administration/profil`
GET	`/administration/profil/edit`
GET, POST	`/administration/roles`
GET, PUT, PATCH, DELETE	`/administration/roles/{id}`
PUT	`/administration/roles/{id}/permissions`
GET	`/administration/roles/matrice-autorisations`
GET	`/administration/roles/{id}/matrice-autorisations`
PUT	`/administration/roles/{id}/autorisations`
GET, POST	`/administration/permissions`
GET, PUT, PATCH, DELETE	`/administration/permissions/{id}`
PUT	`/administration/permissions/{id}/actions`
GET, POST	`/administration/actions`
GET, PUT, PATCH, DELETE	`/administration/actions/{id}`
GET, POST	`/administration/menus`
GET, PUT, PATCH, DELETE	`/administration/menus/{id}`

13. Sécurité et règles d'intégration

- Le rôle `ADMIN` contourne les contrôles de permission métier.
- Les autres rôles doivent posséder `COMPTE\_GERER`, `ROLE\_GERER`, `PERMISSION\_GERER`, `ACTION\_GERER` ou `MENU\_GERER` selon le module.
- Les tableaux `permission\_ids` et `action\_ids` remplacent entièrement les associations existantes.
- Un tableau vide retire toutes les associations.
- Ne jamais stocker le mot de passe ou l'OTP dans le navigateur.

- Conserver le token dans un stockage adapté au niveau de sécurité du frontend.
- Toujours traiter les réponses `401`, `403`, `422` et `429`.

14. Workflow rôles, permissions, menus et actions

1. Créer les actions métier (VOIR, AJOUTER, MODIFIER, SUPPRIMER, IMPRIMER, TELECHARGER) avec POST /administration/actions. Le code est généré par l'API.
2. Créer une permission avec POST /administration/permissions. Son code est généré par l'API et ses actions peuvent être associées avec actionids.
3. Créer un menu avec POST /administration/menus, puis associer les permissions et actions avec permissionids et actionids. Le code menu est généré par l'API.
4. Créer le rôle avec POST /administration/roles. Le champ code du rôle est saisi par le frontend.
5. Charger la matrice avec GET /administration/roles/matrice-autorisations.
6. Enregistrer les cases cochées avec PUT /administration/roles/{id}/autorisations.
7. Affecter le rôle à un utilisateur avec idrole lors de la création ou de la modification du compte.
8. Après connexion, charger GET /navigation/sidebar. Seuls les menus et actions autorisés sont retournés.

15. Déploiement automatique cPanel

Le workflow `.github/workflows/deploy.yml` s'exécute à chaque push sur main. Il lance tous les tests, installe les dépendances de production puis transfère l'application par FTP vers cPanel.

Secrets GitHub requis :

Secret	Exemple
FTP_HOST	ftp.severinzran.ci
FTP_USERNAME	utilisateur FTP cPanel
FTP_PASSWORD	mot de passe FTP cPanel
FTP_REMOTE_PATH	/api-ebac.severinzran.ci/

Le fichier `.env`, les tests, le dossier storage et les fichiers temporaires ne sont pas transférés. Après une migration ajoutée au projet, exécuter `php artisan migrate --force` depuis le terminal cPanel si l'hébergement ne fournit pas de mécanisme automatique pour cette commande.